PEEK-TRANSACTION

Fraud Rule Engine with Reviewer Console

BDML PROJECT | AI23521 – Build and Deployment of Machine Learning Applications

PROJECT OVERVIEW

Peek-Transaction scores every incoming payment transaction for fraud risk, saves the result with its reasons, alerts the team by email when the risk is high, and lets human reviewers mark flagged transactions as reviewed or cleared. Scoring is done by independent, pluggable rules (velocity, unusual amount, impossible travel) that are discovered from a folder, so new fraud patterns can be added without editing the core engine.

1. Problem Statement

Payment fraud is often caught too late because suspicious transactions are buried among normal ones and fraud patterns keep changing. Fixed systems are hard to extend: adding one new check usually means changing core code. Peek-Transaction addresses this with a modular scoring engine, persistent flags, instant alerts, and a review console with role-based access and an audit trail.

2. Project Objectives

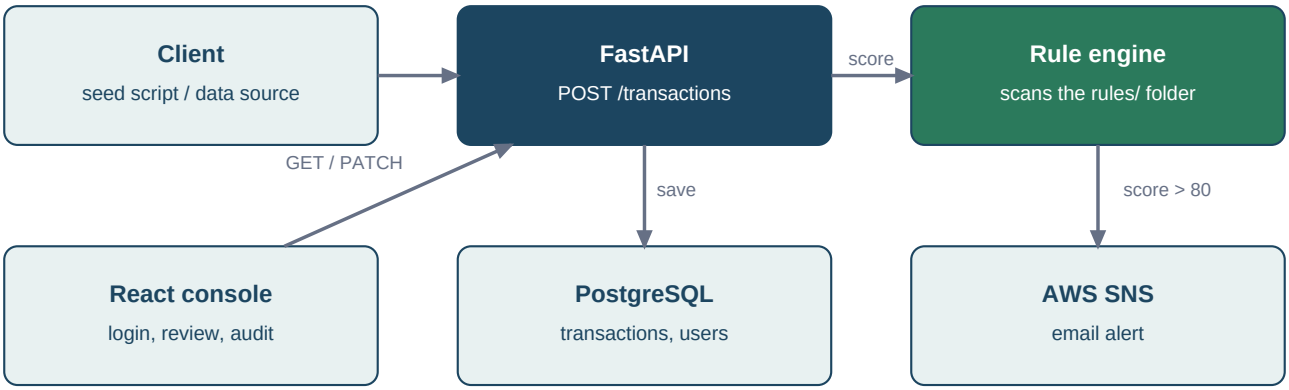
- Evaluate each transaction for risk with at least three independent rules (velocity, unusual amount, impossible location).
- Allow new rules to be added without modifying the core engine.
- Persist all transactions and fraud flags in a database (PostgreSQL).
- Show flagged transactions in a React console where reviewers mark them reviewed or cleared.
- Send an AWS SNS notification when a high-risk threshold is crossed.
- Control who can do what (viewer / analyst / admin) with access rules, and record who reviewed what.
- Verify behaviour with automated tests and repeatable demo scenarios.

3. Decision Models

Component	Input and role	Expected output
Velocity rule	The user's recent transactions; counts those in the 5 minutes before the new one.	+40 when more than 5 transactions fall in 5 minutes.
Unusual amount rule	The user's earlier amounts (needs at least 3).	+50 when the amount is over 5x the user's average.
Impossible location rule	Previous transaction's location and time; Haversine distance and travel speed.	+90 when implied speed exceeds 900 km/h.
Account takeover rule (drop-in demo)	Device ids, cities, amounts and hours from the user's history.	+60 for a new device with one anomaly, +90 with two or more; 0 for a new device alone.

Important: these are deterministic, rule-based scores. No machine-learning model is trained in the current implementation, and the results are not presented as machine-learning predictions. The rule scores and the review decisions stored in the database (reviewed or cleared, with who and when) form a labelled history that could later be used to train a classifier. This is listed as future work.

4. System Architecture



Rules run: velocity, unusual amount, impossible location, plus drop-in rules (for example account takeover).

5. Proposed Workflow

1. Transaction in — a client posts user, amount, city, coordinates, optional timestamp and device id.

2. History — the API loads that user's earlier transactions, oldest first.

3. Scoring — the engine scans the *rules/* folder, runs every rule, adds up the scores and collects reasons.

4. Persistence — score, reasons (JSON) and flag are saved; anything scoring above 0 is flagged.

5. Alert — above the threshold (default 80) a background task publishes to AWS SNS; failures are logged and never break the request.

6. Review — reviewers sign in, see the flagged queue (refreshed every 5 seconds), and mark rows reviewed or cleared. Access rules decide who may do each action, and the reviewer and time are recorded.

6. Technology Stack

Backend/API	Python, FastAPI, Uvicorn, Pydantic (REST API)
Rule engine	Plain Python; pkgutil and importlib discover rule classes at runtime
Database	PostgreSQL via SQLAlchemy and psycopg 3 (SQLite fallback for local use)
Alerts	AWS SNS via boto3; credentials from environment variables (python-dotenv)
Security	PBKDF2-hashed passwords, JWT login tokens, role-based access rules
Frontend	React 18 with Vite, plain CSS, light and dark themes
Testing	pytest (39 automated tests), scripted demo scenarios (seed.py)
Deployment	Run locally with Uvicorn and Vite. Docker containerization is not included in the current implementation.

7. Evaluation

The rules are deterministic and there is no labelled real-world dataset, so classification metrics such as precision, recall and F1 are **not reported**. Instead the system is evaluated in two ways, both run on the implemented code.

Automated tests: 39 pytest tests passed (23 for the rules and engine, including boundary cases and adding a new rule file with no engine change; 16 for login, roles, access rules and the audit trail). No test uses the network or AWS, and the API tests use an in-memory database.

Scenario validation (synthetic data): seven scripted scenarios were posted through the API. The score of the final transaction in each matched the expected value.

Scenario	User	What happens	Expected	Observed
normal	ravi	5 ordinary purchases in Chennai	0	0
velocity	arun	8 transactions within 3 minutes	40	40
amount	meena	4 purchases near Rs.1000, then Rs.150,000	50	50
location	priya	Chennai, then London 15 minutes later	90	90
combined	karthik	normal history, then Rs.150,000 from London 15 minutes after Chennai	140	140
ato (rule loaded)	divya	new device, new city, 4.5x average amount	90	90
ato (rule not loaded)	divya	same attack with only the 3 built-in rules	0	0
newphone	sanjay	new device, same city, normal amount and hour	0	0

The "ato" pair shows the extensibility requirement: the attack scores 0 with the three built-in rules, and scores 90 once *account_takeover.py* is copied into the *rules/* folder, with *engine.py* unchanged. The "newphone" row shows a genuine new device is not punished.

8. Expected Outcome and Limitations

The outcome is a working prototype: transactions are scored, flagged, stored, alerted on and reviewed through one console. It is a decision-support aid for fraud analysts, not an automatic block or a guarantee of fraud detection.

- Thresholds (5 per 5 minutes, 5x average, 900 km/h) are fixed and were not tuned on real data.
- Results above come from synthetic scenarios, not production traffic.
- *POST /transactions* is open (no API key) so that the seed script and data sources can call it; securing ingestion is future work.
- Login tokens are kept in the browser's local storage, which is acceptable for a prototype.
- Future work: train a classifier on labelled review outcomes, add API-key ingestion, and containerize with Docker.

9. Screenshots of the Application

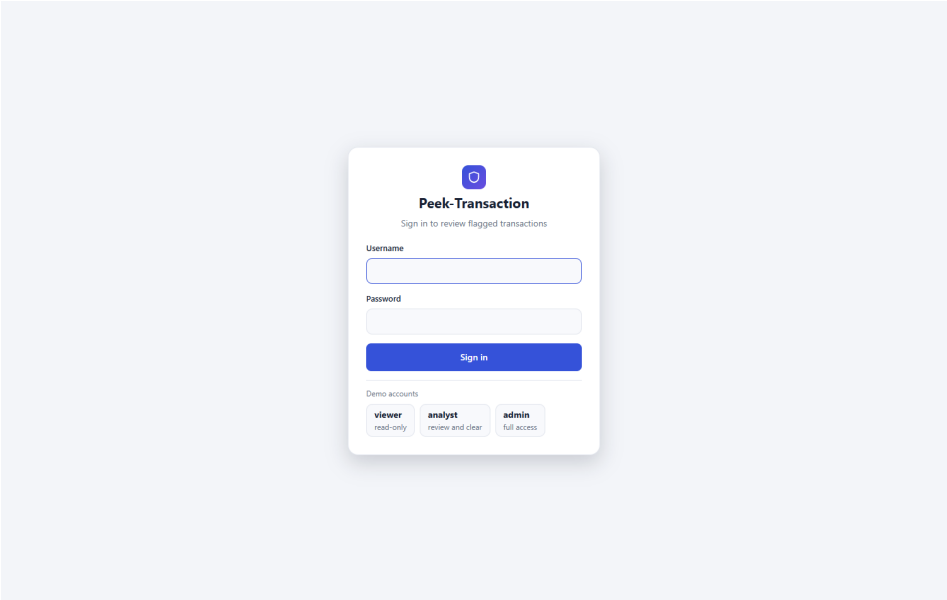


Figure 1. Sign-in page with the demo accounts (viewer, analyst, admin).

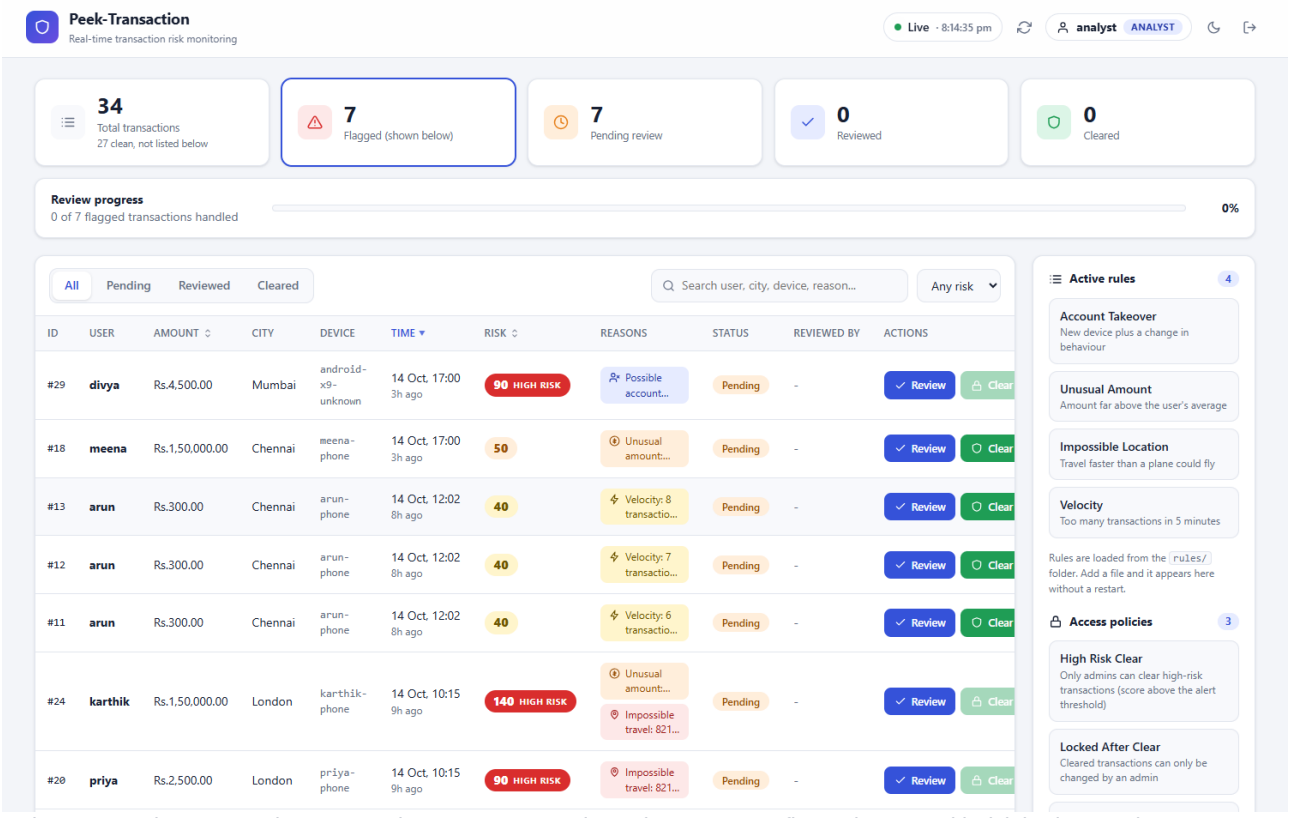


Figure 2. Reviewer console as an analyst: summary cards, review progress, flagged queue with risk badges and reason tags, Active rules (4, including the live-added account takeover rule) and Access policies.

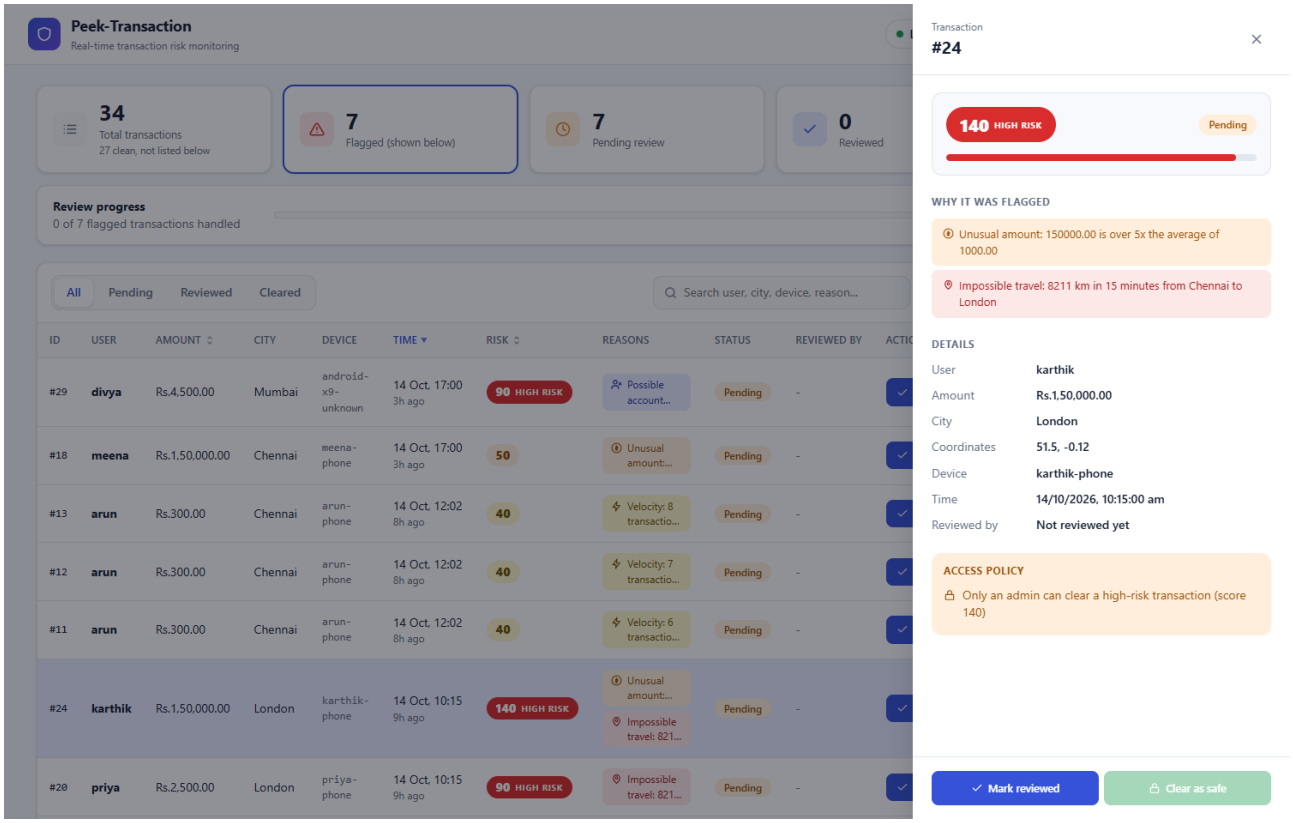


Figure 3. Transaction detail drawer for a score-140 row: both reasons, the access policy that disables "Clear as safe" for an analyst, and the audit field.

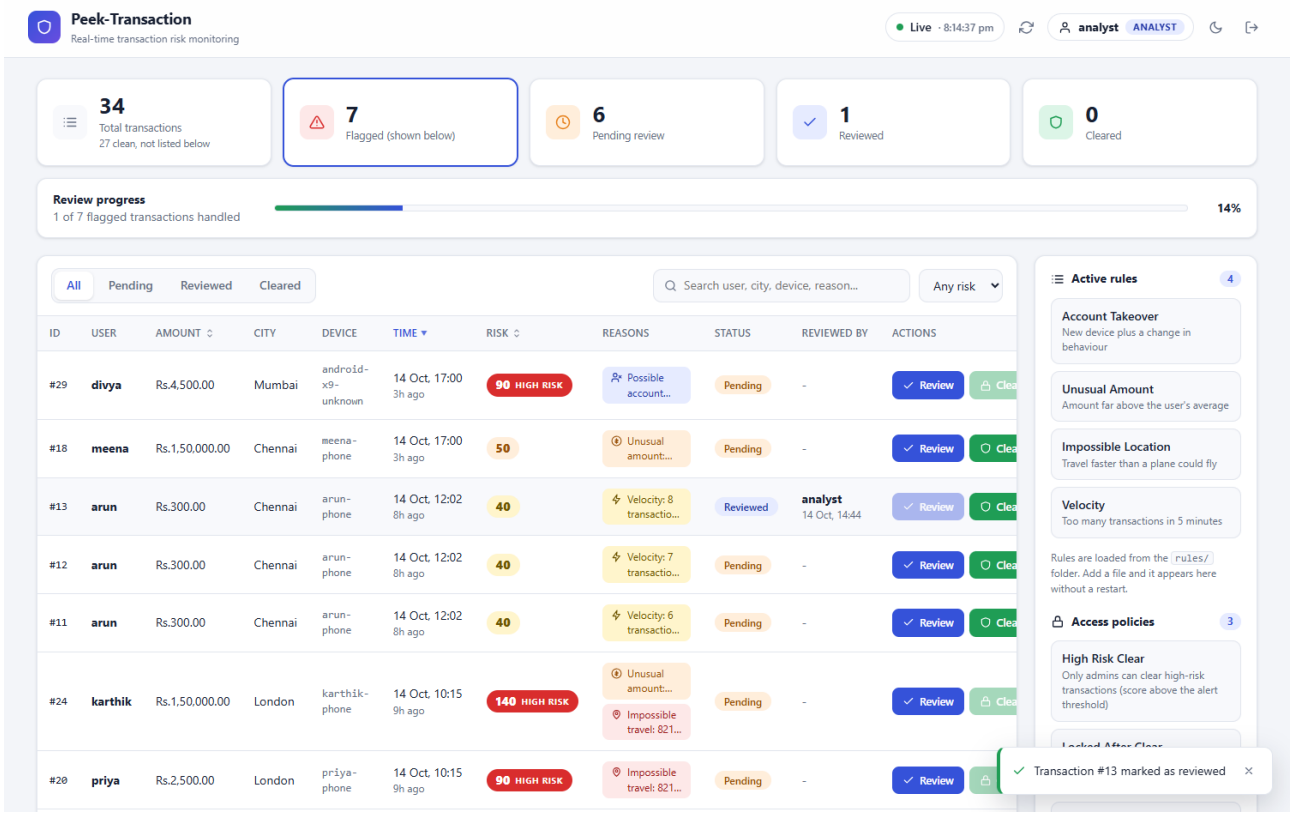


Figure 4. After an analyst marks a row reviewed, the "Reviewed by" column records who and when.

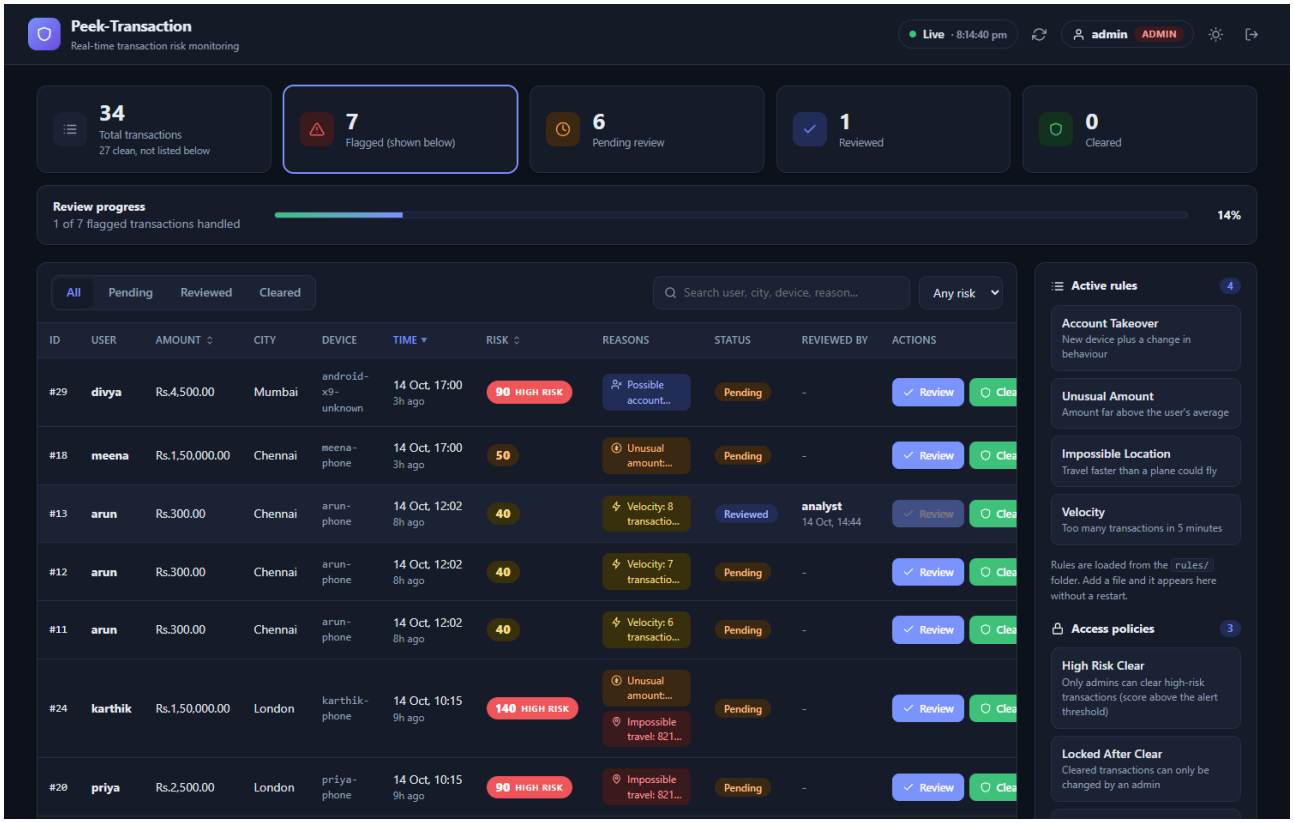


Figure 5. Admin view in dark mode; admins can clear high-risk transactions.